

Assignment 4: Suffix Array Construction via Prefix Doubling with Radix Sort

Release date: March 21, 2026

Due date: March 31, 2026. (Before midnight)

In this assignment, you will implement an algorithm to construct the suffix array of a string using the prefix doubling technique. Your implementation must achieve $O(n \log n)$ time complexity and $O(n)$ space complexity.

Input

The input consists of a single non-empty string containing only lowercase English letters (a–z). The string is provided through standard input.

Output

Print the suffix array of the input string to standard output. The suffix array is the sequence of starting indices of all suffixes of the string, listed in lexicographic order and separated by spaces.

Requirements

- The indices of suffixes start from 0.
- Append a sentinel character '\$' (assumed lexicographically smaller than all input characters) to the input string.
- Use the prefix doubling algorithm covered in class.
- Implement stable counting sort and radix sort explicitly. Stable counting sort can be regarded as a special case of bucket sort where each bucket has only one key value.
- Do not use comparison-based sorting (e.g., `sort()`, `sorted()`).
- You may use Python lists as array-like structures.
- Implement the required function: `def build_suffix_array(s: str) -> list[int]:`
- We will call your program by command line: `python assn3.py`. Your program will read the input string from standard input, print the suffix array to the standard output, and then quit.

Algorithm Overview

1. Sort suffixes based on substrings of length $L = 1, 2, 4, \dots$
2. Use stable counting sort for $L = 1$.
3. Maintain equivalence classes (names) for substrings.
4. Sort pairs $(N[i], N[i+L])$ using radix sort (two counting sort passes).
5. Double L in each iteration.

6. Repeat until all substrings are distinct or $L \geq n$.

Output Format

A single line of space-separated integers representing the suffix array.

Example

Input: banana

Output: 6 5 3 1 0 4 2

Deliverables

1. Source Code

- Complete implementation
- Well-structured and commented

2. Short Report (1–2 pages)

- Explanation of prefix doubling
- Explanation of radix sort and counting sort
- Explanation of renaming (equivalence classes)
- Time complexity analysis
- Discussion of edge cases